

Lexical Grammar

Logan Thomas

February 25, 2026

1 Ambiguity

Ambiguity is recognized between some of the regexes listed below. This ambiguity is solved in the lexer by taking the longest match or through regex precedence. The longest match is to pick the regex that matches the most characters (greedy matching), while the precedence resolution is to define a precedence of regexes such that in the case where two regexes match the same span one is preferred.

1.1 Examples

Some examples of ambiguous matches include:

- "else" matches both $\langle \textit{else} \rangle$ and $\langle \textit{ident} \rangle$. Solved with precedence.
- "1.2e3" matches as either $\langle \textit{float_const} \rangle$ followed by an $\langle \textit{ident} \rangle$ or as a $\langle \textit{scientific} \rangle$. Solved with longest match.
- "r\"abc\"" matches as either an $\langle \textit{ident} \rangle$ followed by a $\langle \textit{string_literal} \rangle$ or as just a $\langle \textit{string_literal} \rangle$. Solved with longest match.

2 Regex

2.1 Substitutions

L	$:=$	$[a - z]$	(1)
U	$:=$	$[A - Z]$	(2)
D	$:=$	$[0 - 9]$	(3)
H	$:=$	$[a - fA - F0 - 9]$	(4)
O	$:=$	$[0 - 7]$	(5)
B	$:=$	$[01]$	(6)

2.2 Identifiers, Literals, and Comments

$_ \{0, 2\} L(? : L U D _)*$	$\rightarrow$	IDENT	(7)
$U(? : L U D)*$	$\rightarrow$	TYPE_IDENT	(8)
$[_ + -]? D +$	$\rightarrow$	NUMERIC_LITERAL (INT_LITERAL) Integer	(9)
$0[xX](H\{1, 16\})$	$\rightarrow$	NUMERIC_LITERAL (RAW_BITS) Hex	(10)
$0[oO](1O\{21\} O\{1, 21\})$	$\rightarrow$	NUMERIC_LITERAL (RAW_BITS) Octal	(11)
$0[bB](B\{1, 64\})$	$\rightarrow$	NUMERIC_LITERAL (RAW_BITS) Binary	(12)
$[_ + -]?(? : D + _.D * _.D +)$	$\rightarrow$	NUMERIC_LITERAL (FLOAT_LITERAL) Float	(13)
$([_ + -]?(? : D + (? : _.D *)? _.D +))[eE]([_ + -]? D +)$	$\rightarrow$	NUMERIC_LITERAL (FLOAT_LITERAL) Scientific	(14)
$(r?)"((? : [^ " \backslash] \backslash \backslash ")*)"$	$\rightarrow$	STRING_LITERAL	(15)
$\# . * \$$	$\rightarrow$	COMMENT	(16)

2.3 Symbols

:	$\rightarrow$	COLON	(17)
,	$\rightarrow$	COMMA	(18)
{	$\rightarrow$	LEFT_CURLY	(19)
(	$\rightarrow$	LEFT_PAREN	(20)
[	$\rightarrow$	LEFT_SQUARE	(21)
}	$\rightarrow$	RIGHT_CURLY	(22)
)	$\rightarrow$	RIGHT_PAREN	(23)
]	$\rightarrow$	RIGHT_SQUARE	(24)
;	$\rightarrow$	SEMI	(25)

2.4 Keywords

and	→	AND	(26)
bool	→	BOOL	(27)
break	→	BREAK	(28)
class	→	CLASS	(29)
elif	→	ELIF	(30)
else	→	ELSE	(31)
enum	→	ENUM	(32)
exit	→	EXIT	(33)
false	→	FALSE	(34)
FLOAT32_MAX	→	FLOAT32_T_MAX	(35)
FLOAT32_MIN	→	FLOAT32_T_MIN	(36)
float32	→	FLOAT32_T	(37)
FLOAT64_MAX	→	FLOAT64_T_MAX	(38)
FLOAT64_MIN	→	FLOAT64_T_MIN	(39)
float64	→	FLOAT64_T	(40)
NaN	→	FLOAT_NAN	(41)
for	→	FOR	(42)
func	→	FUNCTION	(43)
if	→	IF	(44)
INT16_MAX	→	INT16_MAX	(45)
INT16_MIN	→	INT16_MIN	(46)
int16	→	INT16	(47)
INT32_MAX	→	INT32_MAX	(48)
INT32_MIN	→	INT32_MIN	(49)
int32	→	INT32	(50)
INT64_MAX	→	INT64_MAX	(51)
INT64_MIN	→	INT64_MIN	(52)
int64	→	INT64	(53)
INT8_MAX	→	INT8_MAX	(54)
INT8_MIN	→	INT8_MIN	(55)
int8	→	INT8	(56)
iface	→	INTERFACE	(57)
in	→	IN	(58)
mut	→	MUTABLE	(59)
NINF	→	NEG_INF	(60)
not	→	NOT	(61)
or	→	OR	(62)
INF	→	POS_INF	(63)
private	→	PRIVATE	(64)
public	→	PUBLIC	(65)
return	→	RETURN	(66)
string	→	STRING	(67)

true	→	TRUE	(68)
UINT16_MAX	→	UINT16_MAX	(69)
UINT16_MIN	→	UINT16_MIN	(70)
uint16	→	UINT16	(71)
UINT32_MAX	→	UINT32_MAX	(72)
UINT32_MIN	→	UINT32_MIN	(73)
uint32	→	UINT32	(74)
UINT64_MAX	→	UINT64_MAX	(75)
UINT64_MIN	→	UINT64_MIN	(76)
uint64	→	UINT64	(77)
UINT8_MAX	→	UINT8_MAX	(78)
UINT8_MIN	→	UINT8_MIN	(79)
uint8	→	UINT8	(80)
while	→	WHILE	(81)

2.5 Operators

& =	→	AMP_EQUAL	(82)
&	→	AMP	(83)
^^ =	→	CARET_CARET_EQUAL	(84)
^^	→	CARET_CARET	(85)
^ =	→	CARET_EQUAL	(86)
^	→	CARET	(87)
.	→	DOT	(88)
==	→	EQUAL_EQUAL	(89)
=	→	EQUAL	(90)
! =	→	EXCLAIM_EQUAL	(91)
/ =	→	FSLASH_EQUAL	(92)
// =	→	FSLASH_FSLASH_EQUAL	(93)
//	→	FSLASH_FSLASH	(94)
/	→	FSLASH	(95)
< =	→	LANGLE_EQUAL	(96)
<<	→	LANGLE_LANGLE	(97)
<	→	LANGLE	(98)
- =	→	MINUS_EQUAL	(99)
--	→	MINUS_MINUS	(100)
-	→	MINUS	(101)
% =	→	PERCENT_EQUAL	(102)
%	→	PERCENT	(103)
=	→	PIPE_EQUAL	(104)
	→	PIPE	(105)
+ =	→	PLUS_EQUAL	(106)
++	→	PLUS_PLUS	(107)
+	→	PLUS	(108)
?	→	QUESTION	(109)
> =	→	RANGLE_EQUAL	(110)
>>	→	RANGLE_RANGLE	(111)
>	→	RANGLE	(112)
* =	→	STAR_EQUAL	(113)
*	→	STAR	(114)
~ =	→	TILDE_EQUAL	(115)
~	→	TILDE	(116)